

Clase 222 — IAM en la nube: identidades, roles y permisos

Parte: **10 — Seguridad en la nube y contenedores** · Fuente: *AWS IAM User Guide* y *AWS Well-Architected: Identity and Access Management* 🕒 Duración estimada: **120 min** · Nivel: **Intermedio**

🎯 Objetivo

Dominar la gestión de identidad y acceso (IAM) como el verdadero perímetro de la nube: cómo se modelan usuarios, grupos, roles y políticas; cómo se evalúa una petición; y cómo aplicar privilegio mínimo evitando las rutas de escalada de privilegios que buscan los atacantes.

📖 Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Modelar** identidades humanas y de máquina con roles asumibles en lugar de claves estáticas.
2. **Escribir** políticas IAM basadas en el principio de privilegio mínimo.
3. **Explicar** la lógica de evaluación (deny explícito > allow > deny implícito).
4. **Detectar** permisos peligrosos que permiten escalada (`iam:PassRole` , `iam:CreatePolicyVersion` , etc.).
5. **Auditar** una cuenta buscando credenciales inactivas y permisos excesivos.

📖 Temas

#	Tema	Por qué importa
1	Principals: usuarios, grupos, roles	Base del modelo de acceso
2	Políticas de identidad vs de recurso	Definen permisos desde dos lados
3	Roles asumibles y STS	Credenciales temporales en lugar de claves largas
4	Lógica de evaluación de permisos	Predecir si una acción se permite o deniega
5	Privilegio mínimo y límites de permisos	Reducen el blast radius
6	Escalada de privilegios IAM	Rutas que convierten un permiso menor en admin
7	Federación y SSO	Centralizar identidades corporativas

📖 Definiciones y características

- **Usuario IAM:** identidad de larga duración con credenciales propias. *Clave:* preferir roles; los usuarios con claves estáticas son un riesgo si se filtran.
- **Rol IAM:** identidad asumible que otorga credenciales temporales vía STS. *Clave:* sin secreto permanente, ideal para servicios y federación.

- **Política:** documento JSON con `Effect`, `Action`, `Resource` y `Condition`. *Clave:* define permisos de forma declarativa.
- **Policy de recurso:** adjunta al recurso (bucket, cola) en vez de a la identidad. *Clave:* permite acceso cross-account controlado.
- **Permission boundary:** techo máximo de permisos que una identidad puede tener. *Clave:* delega administración sin poder escalar.
- **sts:AssumeRole**: acción que cambia de identidad. *Clave:* base de la federación y de muchas rutas de escalada.
- **Escalada de privilegios:** aprovechar un permiso para obtener otro mayor. *Clave:* permisos como `iam:PassRole` + `ec2:RunInstances` equivalen a admin.

Herramientas y preparación

- CLI del proveedor (`aws`, `az`, `gcloud`) con una cuenta de laboratorio.
- **Prowler** y **ScoutSuite** para auditar IAM (se profundiza en la clase 231).
- **Pacu** (framework de explotación AWS) y **PMapper** para grafos de escalada IAM. Úsalos **solo** en tu propia cuenta de laboratorio.

```
# Enumerar usuarios y sus claves de acceso (AWS)
aws iam list-users
aws iam list-access-keys --user-name lab-user
# Simular si una acción está permitida
aws iam simulate-principal-policy \
  --policy-source-arn arn:aws:iam::123456789012:user/lab-user \
  --action-names s3:DeleteBucket
```

Laboratorio guiado

 Contenido con componente ofensivo. Ejecuta todo **exclusivamente en tu propia cuenta de laboratorio**.

1. Crea un usuario `lab-user` sin permisos y un rol `lab-admin-role` con `AdministratorAccess`.
2. Adjunta a `lab-user` una política que solo permita `iam:PassRole` y `ec2:RunInstances`.
3. Con **PMapper**, genera el grafo de la cuenta: `pmapper graph create` y luego `pmapper query "who can do iam:* with *"`. Observa cómo el usuario aparente-mínimo puede escalar a admin lanzando una instancia con el rol admin.
4. Reproduce la ruta manualmente: lanza una instancia EC2 pasando `lab-admin-role` y obtén sus credenciales temporales desde el servicio de metadatos.
5. **Mitiga:** aplica un *permission boundary* a `lab-user` y añade una `Condition` que restrinja qué roles puede pasar (`iam:PassedToService`). Repite el ataque y verifica que ahora falla.
6. Audita la cuenta con `prowler aws -c iam_*` y revisa hallazgos de claves inactivas y políticas con comodín `*`.
7. Elimina las credenciales de larga duración y sustitúyelas por roles asumibles.

Ejercicios

1. Escribe una política que permita listar un bucket concreto pero denegar borrar objetos.

2. Convierte una integración basada en clave estática en un rol asumido por un servicio.
3. Explica el resultado de una petición con un `Deny` explícito y un `Allow` simultáneos.
4. Añade una `Condition` que exija MFA para acciones destructivas.
5. Usa PMapper para encontrar todas las identidades que pueden asumir un rol admin.
6. Diseña un esquema de permission boundaries para un equipo que autoadministra sus recursos.

Reto verificable

Parte de una política demasiado amplia (`"Action": "*" , "Resource": "*" )` y refactorízala a privilegio mínimo para un caso de uso concreto (una app que lee de una tabla y escribe en una cola).

Criterio de aceptación: la política final solo incluye las acciones estrictamente necesarias sobre los ARNs concretos, incluye al menos una `Condition`, y `simulate-principal-policy` confirma que las acciones legítimas se permiten y una acción no relacionada se deniega.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>AccessDenied</code> inesperado	SCP u organización deniega arriba; revisa políticas heredadas y deny explícitos.
Política con <code>"Action": "*" </code> en producción	Privilegio excesivo; refactoriza a acciones concretas y usa Access Analyzer.
Claves de acceso en el código	Credenciales estáticas filtrables; migra a roles y rota/revoca de inmediato.
<code>iam:PassRole</code> sin condición	Ruta de escalada; restringe con <code>Condition</code> sobre servicio y rol destino.
Usuarios inactivos con claves activas	Falta de revisión; deshabilita credenciales sin uso > 90 días.

Preguntas frecuentes

? ¿Rol o usuario para una aplicación? Siempre rol. Un rol da credenciales temporales que rotan solas; una clave de usuario es un secreto permanente que, si se filtra, sirve al atacante indefinidamente.

? ¿Qué pasa si una política de identidad permite algo y una de recurso lo deniega? Gana el deny. La evaluación combina todas las políticas aplicables y cualquier deny explícito prevalece sobre cualquier allow.

? ¿Cómo aplico privilegio mínimo sin frenar al equipo? Empieza permisivo pero mide: usa Access Analyzer/policy usage para ver qué se usa realmente y recorta lo no utilizado. Itera en vez de adivinar.

Referencias

- AWS IAM User Guide. <https://docs.aws.amazon.com/IAM/latest/UserGuide/>
- AWS — IAM policy evaluation logic. https://docs.aws.amazon.com/IAM/latest/UserGuide/reference_policies_evaluation-logic.html
- PMapper (Principal Mapper). <https://github.com/nccgroup/PMapper>

- Rhino Security Labs — AWS IAM privilege escalation methods.
<https://rhinosecuritylabs.com/aws/aws-privilege-escalation-methods-mitigation/>
- Microsoft Entra ID docs. <https://learn.microsoft.com/entra/identity/>

Siguiente clase

Clase 223 - Seguridad en AWS